RELATÓRIO OFICIAL E GUIA COMPLETO DE ESTUDOS

Curso Técnico em Desenvolvimento de Sistemas | ETEC / CPS

Componente: Desenvolvimento de Sistemas I — Agendas 01 a 11 (Revisão Completa para Avaliação)

Estudante: Iraci Berto Gomes | **Arquivo Oficial:** IraciBertoGomes_Ag11_DSI

1. Introdução: A Jornada de Desenvolvimento de Sistemas I

Este documento foi elaborado para servir como comprovação técnica da Atividade da Agenda 11 e como **Guia Oficial de Estudos para a Avaliação**. A seguir, cada uma das 11 agendas tem sua fundamentação teórica, sintaxe, exemplo prático e dicas de prova apresentadas detalhadamente.

2. Conteúdo Didático das Apostilas (Agendas 01 a 11)

Agenda 01 — Raciocínio Lógico e Algoritmos

• **Teoria:** Um algoritmo é uma sequência finita, ordenada e não ambígua de instruções para solucionar um problema. Antes de codificar em qualquer linguagem, o profissional de TI deve decompor o desafio em etapas lógicas: Entrada de dados (dados iniciais), Processamento (transformações e cálculos) e Saída (resultado apresentado). O pensamento computacional estrutura a solução antes da digitação da primeira linha de código.

```
# Exemplo: Cálculo do volume de abastecimento
capacidade_maxima = 1000  # Entrada: litros
nivel_atual = 350         # Entrada: litros
volume_necessario = capacidade_maxima - nivel_atual  # Processamento
print('Litros para encher:', volume_necessario)       # Saída: 650
```

■■ Dica de Prova: Não confunda 'Dado' com 'Informação'. Dado é o elemento bruto não processado (ex: número 350). Informação é o dado processado com contexto e significado (ex: 'Faltam 650 litros para transbordar').

• **Aplicação na Agenda 11:** Na Agenda 11, o algoritmo sequencia com precisão: limpar tela -> cabeçalho -> iterar níveis -> exibir alertas -> restaurar terminal.

Agenda 02 — Introdução à Programação com Python: Variáveis e Tipos Primitivos

• **Teoria:** Variáveis são espaços alocados na memória do computador para armazenar valores temporários. Python possui tipagem dinâmica (o tipo é inferido automaticamente). Os quatro tipos primitivos fundamentais são: • int: Números inteiros (ex: 1, 5, -10). • float: Números decimais de ponto flutuante (ex: 7.5, 0.05). Usa-se ponto e não vírgula! • str: Cadeias de caracteres (textos) delimitadas por aspas simples ou duplas. • bool: Valores lógicos booleanos, estritamente True ou False (com inicial maiúscula).

```
nivel = 3                          # int
porcentagem = 60.0                  # float
situacao = 'Médio (Estável)'        # str
sistema_ativo = True                # bool
print(type(nivel), type(porcentagem), type(situacao), type(sistema_ativo))
```

■■ Dica de Prova: Nomes de variáveis devem usar 'snake_case' (letras minúsculas com sublinhado). Não podem começar com números, não podem conter espaços nem traços, e diferenciam maiúsculas de minúsculas (case-sensitive)!

• **Aplicação na Agenda 11:** O nível 1 a 5 é trabalhado como int, a situação é uma str e o status da telemetria é controlado como bool.

Agenda 03 — Entrada de Dados, Operadores Aritméticos e f-Strings

• **Teoria:** O comando input() sempre captura dados do teclado no formato de texto (str). Para operações matemáticas, é obrigatório realizar o 'type casting' (conversão de tipo) usando int() ou float(). Operadores Aritméticos: + (adição), - (subtração), * (multiplicação), / (divisão real), // (divisão inteira), % (resto da divisão / módulo), ** (exponenciação). Para exibir resultados com texto e variáveis juntos, o padrão moderno e recomendado do Python são as f-strings (f'Texto {variavel}').

```
nivel_digitado = int('4')  # Simulação de casting seguro
capacidade_maxima = 5
porcentagem = (nivel_digitado / capacidade_maxima) * 100
print(f'Nível informado: {nivel_digitado} | Ocupação: {porcentagem:.1f}%')
```

■■ Dica de Prova: '7' + '3' resulta em '73' (concatenação de strings) e não 10! Se esquecer de converter o input() com int() ou float(), o Python fará concatenação ou gerará TypeError.

• **Aplicação na Agenda 11:** Utilizamos f-strings no comando de exibição formatada do reservatório: f'>> Nível {nivel}: {situacao}'.

Agenda 04 — Controle de Versão com Git (Parte 1: Fundamentos Locais)

• **Teoria:** O Git é um sistema distribuído de controle de versão (VCS) que registra o histórico completo de evolução do código. Ele opera através de três áreas de trabalho locais: 1. Working Directory (Diretório de Trabalho): Arquivos que você está editando. 2. Staging Area / Index: Área de preparação onde você seleciona quais arquivos farão parte do próximo registro. 3. Repository / Local Git (.git): Banco de dados onde os commits são permanentemente gravados.

```
# No terminal dentro da pasta do projeto:
git init
git add IraciBertoGomes_Ag11_DSI.py
git commit -m 'feat: adicionar monitoramento de nivel de agua'
```

■■ Dica de Prova: 'git add' não salva o arquivo no histórico final! Ele apenas prepara (stage). O que grava a versão de forma definitiva é o 'git commit -m "..."'.

• **Aplicação na Agenda 11:** Nossa atividade cumpre a Habilidade 4.2 criando commits semânticos no repositório da pasta AGENDA_11.

Agenda 05 — Controle de Versão com Git e GitHub (Parte 2: Repositórios Remotos)

• **Teoria:** O GitHub é uma plataforma em nuvem que hospeda repositórios Git, viabilizando colaboração e backup seguro. O branch principal moderno é chamado de 'main'. Para sincronizar o computador com a nuvem, vinculamos um 'remote' (apelidado de 'origin') e enviamos os commits com 'push'. Comandos essenciais: git remote add origin , git branch -M main, git push -u origin main, git clone .

```
# Conectar o projeto local ao repositório criado no GitHub:
git remote add origin https://github.com/IraciBerto/Controle_Agua.git
git push -u origin main
```

■■ Dica de Prova: O parâmetro '-u' (upstream) no 'git push -u origin main' cria o vínculo permanente entre o branch local e o remoto. Nos envios seguintes, basta digitar apenas 'git push'.

• **Aplicação na Agenda 11:** A entrega da atividade exige compartilhar o link do repositório GitHub no fichário virtual da ETEC.

Agenda 06 — Estruturas de Decisão Simples e Composta (if / else)

• **Teoria:** Permite que o programa escolha caminhos alternativos de execução com base no teste de uma condição lógica. • Condição Simples: 'if condicao:' -> executa o bloco indentado somente se o teste for True. • Condição Composta: 'if condicao: ... else: ...' -> se True executa o primeiro bloco; se False executa o else. Operadores Relacionais: == (igualdade), != (diferente), > (maior), < (menor), >= (maior ou igual), <= (menor ou igual).

```
nivel_agua = 1
if nivel_agua == 1:
    print('Alerta Crítico!')
else:
    print('Nível Operacional Seguro.')
```

■■ Dica de Prova: Não confunda '=' (atribuição de valor a variável) com '==' (comparação lógica de igualdade)! Escrever 'if nivel = 1:' gera erro de sintaxe imediato (SyntaxError).

• **Aplicação na Agenda 11:** A função de escolha de cores utiliza a estrutura de decisão para identificar qual cor pertence a qual nível.

Agenda 07 — Estruturas de Decisão Encadeadas (elif) e Operadores Lógicos

• **Teoria:** Quando um problema possui três ou mais ramificações possíveis, o uso de 'elif' (abreviação de else if) é a forma mais clara, elegante e eficiente. O Python avalia cada condição de cima para baixo e executa apenas a primeira verdadeira. Operadores Lógicos: • and (E): Verdadeiro somente se TODAS as condições forem verdadeiras. • or (OU): Verdadeiro se PELO MENOS UMA condição for verdadeira. • not (NÃO): Inverte o valor booleano (True vira False, False vira True).

```
nivel = 3
if nivel == 1:
    cor = 'Vermelho'
elif nivel == 2:
    cor = 'Amarelo'
elif nivel == 3:
    cor = 'Verde'
elif nivel == 4:
    cor = 'Ciano'
elif nivel == 5:
    cor = 'Azul'
else:
    cor = 'Branco'
```

■■ Dica de Prova: Usar vários 'if' independentes seguidos faz o Python testar todos eles, mesmo que o primeiro já tenha sido verdadeiro. Usar 'elif' faz o Python parar a checagem assim que encontra o primeiro verdadeiro, economizando processamento!

• **Aplicação na Agenda 11:** É exatamente essa estrutura if/elif/else que constrói a função oficial 'definir_cor_mensagem(nivel)'.

Agenda 08 — Estruturas de Repetição: Laço for e Laço while

• **Teoria:** Estruturas de repetição (loops) automatizam tarefas recorrentes sem necessidade de duplicar linhas de código. • for: Indicado quando conhecemos previamente o número de repetições ou quando percorremos coleções (listas, sequências). • while: Indicado quando a repetição depende de uma condição lógica cujo número de voltas não é fixo a priori. • range(inicio, fim, passo): Gera uma sequência numérica. Importante: o valor 'fim' nunca é incluído (ex: range(1, 6) gera 1, 2, 3, 4, 5).

```
print('Contagem dos 5 níveis do reservatório:')
for n in range(1, 6):
    print(f'Testando sensor do nível {n}...')
print('Sensores operacionais!')
```

■■ Dica de Prova: No while, se você esquecer de atualizar a variável de controle dentro do laço (ex: contador += 1), o programa entrará em 'Loop Infinito' e travará o computador.

• **Aplicação na Agenda 11:** O procedimento 'simular_monitoramento' utiliza o laço for para iterar pelos 5 níveis do reservatório.

Agenda 09 — Estruturas de Dados: Listas em Python (Coleções e Índices)

• **Teoria:** Uma lista é uma estrutura de dados que armazena múltiplos valores organizados em sequência sob um único nome de variável. Características fundamentais ensinadas na Agenda 09: 1. Declarada com colchetes: lista = [item1, item2, item3]. 2. Indexação base zero: O primeiro elemento SEMPRE está na posição [0], o segundo na [1], etc. 3. Nomes no plural: Boas práticas da ETEC exigem nomes expressivos no plural (ex: 'niveis', 'mensagens', 'alunos'). 4. A função len(): Retorna a quantidade total de elementos na lista (ex: len(niveis) é 5). 5. Percurso com for: Pode ser feito diretamente ('for item in lista:') ou por índice ('for i in range(len(lista)):').

```
niveis = [1, 2, 3, 4, 5]
mensagens = ['Crítico', 'Baixo', 'Médio', 'Alto', 'Alerta Máximo']
for i in range(len(niveis)):
    print(f'Posição {i} -> Nível {niveis[i]}: {mensagens[i]}')
```

■■ Dica de Prova: Se uma lista tem 5 elementos, os índices vão de 0 a 4! Tentar acessar 'niveis[5]' gera o clássico erro 'IndexError: list index out of range'.

• **Aplicação na Agenda 11:** O enunciado exige explicitamente representar os níveis ou mensagens em uma lista. Utilizamos duas listas paralelas: niveis e mensagens.

Agenda 10 — Modularização: Funções com Retorno e Procedimentos (SRP)

• **Teoria:** Funções dividem problemas complexos em pequenos blocos reutilizáveis com a palavra-chave 'def'. A grande lição da Agenda 10 é a distinção entre Procedimentos e Funções: • Procedimento: Executa uma ação visual (print, formatação) e NÃO possui retorno (devolve None). • Função com Retorno (return): Executa cálculos, regras de negócio ou mapeamentos e DEVOLVE o resultado para o programa com 'return'. • Regra de Ouro: 'Exibir serve para informar; Retornar serve para reutilizar'. • Princípio da Responsabilidade Única (SRP): Cada função deve fazer apenas UMA coisa bem feita.

```
def definir_cor(nivel):
    if nivel == 1: return 'Vermelho'
    elif nivel == 2: return 'Amarelo'
    elif nivel == 3: return 'Verde'
    elif nivel == 4: return 'Ciano'
    elif nivel == 5: return 'Azul'
    else: return 'Branco'

cor_obtida = definir_cor(1)  # O valor retornado é guardado na variável
print('Cor calculada pela função:', cor_obtida)
```

■■ Dica de Prova: O comando 'return' ENCERRA a execução da função imediatamente! Qualquer código escrito após o 'return' dentro do mesmo bloco é inalcançável (dead code).

• **Aplicação na Agenda 11:** A exigência do enunciado 'Criar uma função responsável por definir a cor da mensagem conforme o nível informado' foi atendida com return!

Agenda 11 — Bibliotecas Padrão e Externas: PIP e Colorama no Terminal

• **Teoria:** Bibliotecas são conjuntos de funções e recursos prontos que evitam 'reinventar a roda'. 1. Bibliotecas Padrão (nativas do Python): • os: Interação com o sistema operacional (ex: os.system('cls')). • sys: Acesso a variáveis do ambiente Python e configuração de UTF-8. • time: Controle de tempo e pausas realistas (time.sleep(segundos)). • math / random: Funções matemáticas e sorteios aleatórios. 2. O Gerenciador PIP: Ferramenta de linha de comando para instalar bibliotecas externas: 'pip install colorama'. 3. Biblioteca Colorama: Permite colorir terminais de forma multiplataforma: • Fore: Cor do texto (Fore.RED, Fore.YELLOW, Fore.GREEN, Fore.CYAN, Fore.BLUE). • Style.RESET_ALL: Restaura o estilo padrão do terminal após a exibição das mensagens.

```
from colorama import Fore, Style
import time

print(Fore.YELLOW + 'Aguardando sensor...' + Style.RESET_ALL)
time.sleep(1)  # Pausa de 1 segundo
print(Fore.GREEN + 'Sensor calibrado com sucesso!' + Style.RESET_ALL)
```

■■ Dica de Prova: Por que 'Style.RESET_ALL' é obrigatório? Porque os códigos ANSI são persistentes! Se você não resetar, todos os prints subsequentes e o próprio terminal continuarão na última cor usada.

• **Aplicação na Agenda 11:** O núcleo visual da atividade foi estruturado com colorama e pausas com time.sleep() para simular telemetria em tempo real.

3. O Projeto Integrador da Agenda 11: Controle de Níveis de Água

A atividade consolida os conteúdos em uma simulação de monitoramento hidráulico executada no terminal:

Nível	Situação Operacional	Cor Sugerida	Código Colorama	Diagnóstico Técnico
Nível 1	Muito baixo (crítico)	Vermelho	`Fore.RED`	Risco iminente de desabastecimento.
Nível 2	Baixo	Amarelo	`Fore.YELLOW`	Atenção para acionamento de reposição.
Nível 3	Médio	Verde	`Fore.GREEN`	Volume ideal para operação regular.
Nível 4	Alto	Ciano	`Fore.CYAN`	Nível seguro com margem satisfatória.
Nível 5	Muito alto (alerta)	Azul	`Fore.BLUE`	Risco iminente de transbordamento.

Código-Fonte Oficial (IraciBertoGomes_Ag11_DSI.py)

```
"""
================================================================================
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS - ETEC / CPS
Componente Curricular: Desenvolvimento de Sistemas I (DS I)
Atividade da Agenda 11: Controle de Níveis de Água no Reservatório
Estudante: Iraci Berto Gomes
Arquivo de Entrega: IraciBertoGomes_Ag11_DSI.py

RECURSOS EXIGIDOS E APLICADOS NESTE SCRIPT:
1. FOR: Estrutura de repetição para percorrer as listas de medição.
2. LISTAS: Coleções ordenadas armazenando os 5 níveis e mensagens.
3. BIBLIOTECAS:
- math: Biblioteca padrão para cálculos de volume e capacidade (pi, pow).
- random: Biblioteca padrão para simulação aleatória de oscilação de nível (randint, choice).
- time: Biblioteca padrão para pausas temporais entre leituras (sleep).
- os: Biblioteca padrão para comando de limpeza do console (system).
- sys: Biblioteca padrão para configuração de suporte UTF-8.
- colorama: Biblioteca externa para coloração das mensagens e restauração do terminal.
================================================================================
"""

# ------------------------------------------------------------------------------
# AGENDA 11: IMPORTAÇÃO DAS BIBLIOTECAS (Padrão e Externas)
# ------------------------------------------------------------------------------
import os # Biblioteca padrão: limpeza de terminal (os.system)
import sys # Biblioteca padrão: codificação UTF-8
import time # Biblioteca padrão: pausas realistas (time.sleep)
import math # Biblioteca padrão: cálculos matemáticos de volume (math.pi, math.pow)
import random # Biblioteca padrão: sorteio/simulação de sensores (random.choice, random.randint)
from colorama import Fore, Style # Biblioteca externa: cores e restauração de terminal

# Configuração de suporte a UTF-8 no terminal Windows para acentuação perfeita
if sys.platform == 'win32':
try:
sys.stdout.reconfigure(encoding='utf-8')
except AttributeError:
pass


# ------------------------------------------------------------------------------
# AGENDA 10 & 11: PROCEDIMENTOS DE INTERFACE E CABEÇALHO
# ------------------------------------------------------------------------------

def limpar_tela():
"""Limpa o terminal de forma multiplataforma (Agenda 10 e 11)."""
os.system('cls' if os.name == 'nt' else 'clear')


def exibir_cabecalho():
"""Exibe o cabeçalho oficial do sistema com formatação colorama (Agenda 11)."""
print(Style.BRIGHT + Fore.CYAN + "=" * 68)
print(" SISTEMA DE MONITORAMENTO DE NÍVEIS DE ÁGUA - RESERVATÓRIO")
print(" Telemetria e Controle de Risco Operacional")
print("=" * 68 + Style.RESET_ALL)
print()


# ------------------------------------------------------------------------------
# AGENDA 11: FUNÇÃO UTILIZANDO A BIBLIOTECA MATH
# ------------------------------------------------------------------------------

def calcular_capacidade_reservatorio(raio_metros, altura_metros):
"""
Função com retorno (Agenda 10) que utiliza a biblioteca MATH (Agenda 11):
Calcula a capacidade volumétrica de um reservatório cilíndrico em litros.
Fórmula matemática: Volume = pi * raio² * altura
1 metro cúbico (m³) = 1000 litros.
"""
volume_m3 = math.pi * math.pow(raio_metros, 2) * altura_metros
capacidade_litros = volume_m3 * 1000
return capacidade_litros


# ------------------------------------------------------------------------------
# AGENDA 10 & 11: FUNÇÃO COM RETORNO PARA DEFINIR A COR DA MENSAGEM
```

```
# ------------------------------------------------------------------------
def definir_cor_mensagem(nivel):
"""
Função com retorno (Agenda 10):
Define e retorna o código de cor da biblioteca colorama correspondente
ao nível informado, conforme a tabela de risco da atividade:

- Nível 1: Muito baixo (crítico) -> Vermelho (Fore.RED)
- Nível 2: Baixo -> Amarelo (Fore.YELLOW)
- Nível 3: Médio -> Verde (Fore.GREEN)
- Nível 4: Alto -> Ciano (Fore.CYAN)
- Nível 5: Muito alto (alerta) -> Azul (Fore.BLUE)

Aplica estruturas condicionais if, elif e else (Agendas 06 e 07).
"""
if nivel == 1:
return Fore.RED
elif nivel == 2:
return Fore.YELLOW
elif nivel == 3:
return Fore.GREEN
elif nivel == 4:
return Fore.CYAN
elif nivel == 5:
return Fore.BLUE
else:
return Fore.WHITE


# ------------------------------------------------------------------------
# AGENDA 10 & 11: PROCEDIMENTO PARA EXIBIR A MENSAGEM COLORIDA E RESTAURAR
# ------------------------------------------------------------------------

def exibir_situacao_reservatorio(nivel, situacao):
"""
Procedimento que exibe a situação atual do reservatório com a cor correspondente
e restaura imediatamente o estilo padrão do terminal (Agenda 10 e 11).
"""
cor = definir_cor_mensagem(nivel)
print(cor + Style.BRIGHT + f">> Nível {nivel}: {situacao}" + Style.RESET_ALL)


# ------------------------------------------------------------------------
# AGENDA 08, 09, 10 & 11: SIMULAÇÃO COMPLETA UTILIZANDO FOR, LISTAS E TIME
# ------------------------------------------------------------------------

def simular_ciclo_completo(lista_niveis, lista_mensagens, intervalo_segundos=1):
"""
Procedimento que percorre as LISTAS utilizando o laço FOR (Agendas 08 e 09),
exibindo a progressão completa de risco dos 5 níveis do reservatório.
"""
print(Style.BRIGHT + "ETAPA 1: Diagnóstico Sequencial dos 5 Níveis de Água (Laço FOR e Listas)\n" + Style.RESET_ALL)
print("-" * 68)

# Laço FOR percorrendo as listas pelo índice com len() e range() (Agendas 08 e 09)
for i in range(len(lista_niveis)):
exibir_situacao_reservatorio(lista_niveis[i], lista_mensagens[i])
time.sleep(intervalo_segundos) # Pausa com biblioteca time (Agenda 11)

print("-" * 68)
print()


# ------------------------------------------------------------------------
# AGENDA 11: SIMULAÇÃO DINÂMICA UTILIZANDO A BIBLIOTECA RANDOM
# ------------------------------------------------------------------------

def simular_leitura_aleatoria_sensor(lista_niveis, lista_mensagens):
"""
Procedimento que utiliza a biblioteca RANDOM (Agenda 11) para simular
a leitura em tempo real de um sensor industrial sujeito a oscilações.
Utiliza random.choice() para sortear um nível diretamente da lista.
"""
print(Style.BRIGHT + "ETAPA 2: Teste de Telemetria Dinâmica com Sensor Aleatório (Biblioteca RANDOM)\n" + Style.RESET_ALL)

# Sorteio aleatório de um dos níveis cadastrados na lista (Agenda 11 - random.choice)
```

```
indice_sorteado = random.randint(0, len(lista_niveis) - 1)
nivel_sorteado = lista_niveis[indice_sorteado]
mensagem_sorteada = lista_mensagens[indice_sorteado]

print("Sensor de campo efetuou uma leitura instantânea:")
exibir_situacao_reservatorio(nivel_sorteado, f"{mensagem_sorteada} [Leitura Sorteada via Sensor]")
print()


# ------------------------------------------------------------------------
# PROGRAMA PRINCIPAL
# ------------------------------------------------------------------------

if __name__ == "__main__":
limpar_tela()
exibir_cabecalho()

# 1. Aplicação da biblioteca MATH (Agenda 11):
# Calcula e exibe a capacidade física do reservatório cilíndrico (raio=2.5m, altura=4.0m)
capacidade_total = calcular_capacidade_reservatorio(raio_metros=2.5, altura_metros=4.0)
print(f"Capacidade Total Calculada (math.pi e math.pow): {capacidade_total:,.0f} litros".replace(",", "."))
print("-" * 68)
print()

# 2. Aplicação de LISTAS (Agenda 09):
# Duas listas com nomes no plural armazenando os níveis e as mensagens de alerta
niveis = [1, 2, 3, 4, 5]

mensagens = [
"Muito baixo (crítico) - Risco iminente de desabastecimento",
"Baixo - Atenção para acionamento da captação secundária",
"Médio - Volume ideal para operação regular",
"Alto - Nível seguro com margem de abastecimento satisfatória",
"Muito alto (alerta) - Risco de transbordamento! Vertedouro ativado"
]

# 3. Aplicação do laço FOR e das LISTAS (Agendas 08 e 09):
simular_ciclo_completo(niveis, mensagens, intervalo_segundos=0.8)

# 4. Aplicação da biblioteca RANDOM (Agenda 11):
simular_leitura_aleatoria_sensor(niveis, mensagens)

# 5. Aplicação da biblioteca COLORAMA (Agenda 11) na restauração final:
print(Fore.GREEN + Style.BRIGHT + "[OK] Monitoramento concluído com sucesso!" + Style.RESET_ALL)
print("Estilo padrão do terminal devidamente restaurado com Style.RESET_ALL.\n")
```

Evidência de Execução Real no Console

4. Competências e Habilidades Consolidadas (ETEC / CPS)

- **Competência 4:** Elaborar sistemas aplicando princípios e paradigmas de programação estruturada.
- **Habilidade 4.1:** Codificar programas utilizando decomposição procedural, listas dinâmicas e funções puras.
- **Habilidade 4.2:** Versionar o projeto utilizando ambiente integrado e controle de versão semântico com Git.